COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithm	**SECTION:**	Kenji Baltazar
DATE PERFORMED:	07/28/2026	**DATE SUBMITTED:**	07/30/2026
NAME:	Kenji Baltazar	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 4.1 STACKS

1. PROCEDURE OUTPUT

ILO A: Create a stack using the C++ STL

CODE *(main.cpp)*

```cpp
#include <iostream>
#include <stack> //calling stack from STL


int main(){
    //crete an object for the stack
    std::stack<int> newStack;

    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
    //push to the stack
    newStack.push(3);
    newStack.push(8);
    newStack.push(15);

    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;
    std::cout<<"The size of the stack: "<<newStack.size()<<std::endl;
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    std::cout<<"Removing elements from the stack: \n";
    newStack.pop();
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"The top of the element is: "<<newStack.top()<<std::endl;
    newStack.pop();
    std::cout<<"Stack Empty? "<<newStack.empty()<<std::endl;

    return 0;
}
```

OUTPUT

```
Stack Empty? 1
Stack Empty? 0
The size of the stack: 3
The top of the element is: 15
Removing elements from the stack:
The top of the element is: 8
The top of the element is: 3
Stack Empty? 1
```

ILO B: Develop C++ code that uses both arrays and linked lists to create a stack

B1. CODE *(main2.cpp)*

```cpp
#include <iostream>

//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;

void push();
void pop();
void Top();
bool isEmpty();
bool isFull();

int main(){
    int choice;
    std::cout<<"Enter the number of max elements for the new stack: ";
    std::cin>>i;
    while(true){
        std::cout<<"STACK OPERATIONS"<<std::endl;
        std::cout<<"1.PUSH\n2.POP\n3.TOP\n4.isEmpty"<<std::endl;
```

```
21          std::cin>>choice;
22          switch(choice){
23              case 1:
24                  push();
25                  break;
26              case 2:
27                  pop();
28                  break;
29              case 3:
30                  Top();
31                  break;
32              case 4:
33                  std::cout<<isEmpty()<<std::endl;
34                  break;
35              default:
36                  std::cout<<"Invalid."<<std::endl;
37                  break;
38          }
39      }
40      return 0;
41  }
42
43  bool isEmpty(){
44      //create a condition to check whether the top element is equal to -1
45      if(top == -1) return true;
46      return false;
47  }
48
49  bool isFull(){
50      //check if we reaced the max capacity of the array:
51      if(top > i-1) return true;
52      return false;
53  }
54
55  void push(){
56      //check if full -> if yes return an error
57      if(isFull()){
58          std::cout<<"stack overflow"<<std::endl;
59          return;
60      }
```

```
61
62      std::cout<<"New Value to add into the stack"<<std::endl;
63      std::cin>>newData;
64      //increment the top(index by 1) then assign the new data;
65      stack[++top] = newData;
66  }
67
68  void Top(){
69      //check if the stack is empty
70      if(isEmpty()){
71          std::cout<<"Stack underflow."<<std::endl;
72      }
73
74      std::cout<<"The element on the top of the stack is "<<stack[top]<<std::endl;
75  }
76
77  void pop(){
78      //check if the stack is empty
79          if(isEmpty()){
80          std::cout<<"Stack underflow."<<std::endl;
81      }
82
83      //display the top value
84      std::cout<<"Popping: "<<stack[top]<<std::endl;
85      //decrement the top value from the stack
86      top--;
87  }
```

B1. OUTPUT

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
1
New Value to add into the stack
67
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
1
New Value to add into the stack
68
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
1
New Value to add into the stack
69
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
1
stack overflow
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
3
The element on the top of the stack is 69
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
2
Popping: 69
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
2
Popping: 68
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
2
Popping: 67
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
4
1
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
2
Stack underflow.
```

```
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
5
Invalid.
```

MODIFIED CODE

```cpp
#include <iostream>

//static array, what do we need?
const size_t maxCap = 100;
int stack[maxCap];
int top = -1, i, newData;

void push();
void pop();
void Top();
void printall();
bool isEmpty();
bool isFull();

int main(){
    int choice;
    std::cout<<"Enter the number of max elements for the new stack: ";
    std::cin>>i;
    while(choice != 6){
        std::cout<<"====================="<<std::endl;
        std::cout<<"STACK OPERATIONS"<<std::endl;
        std::cout<<"1.PUSH\n2.POP\n3.TOP\n4.isEmpty\n5.PRINT ALL\n6.EXIT"<<std::endl;
        std::cout<<"====================="<<std::endl;
        std::cin>>choice;
        switch(choice){
            case 1:
                push();
                break;
            case 2:
                pop();
                break;
            case 3:
                Top();
                break;
            case 4:
                std::cout<<isEmpty()<<std::endl;
                break;
            case 5:
                printall();
                break;
```

```
41              default:
42                  if(choice > 6){
43                      std::cout<<"Invalid."<<std::endl;
44                  }
45                  break;
46          }
47          std::cout<<"====================="<<std::endl;
48      }
49      return 0;
50  }
51
52  bool isEmpty(){
53      //create a condition to check whether the top element is equal to -1
54      if(top == -1) return true;
55      return false;
56  }
57
58  bool isFull(){
59      //check if we reaced the max capacity of the array:
60      if(top > i-1) return true;
61      return false;
62  }
63
64  void push(){
65      //check if full -> if yes return an error
66      if(isFull()){
67          std::cout<<"Stack overflow."<<std::endl;
68          return;
69      }
70
71      std::cout<<"New Value to add into the stack"<<std::endl;
72      std::cin>>newData;
73      //increment the top(index by 1) then assign the new data;
74      stack[++top] = newData;
75  }
76
77  void Top(){
78      //check if the stack is empty
79      if(isEmpty()){
80          std::cout<<"Stack underflow."<<std::endl;
```

```
81      }
82
83      std::cout<<"The element on the top of the stack is "<<stack[top]<<std::endl;
84  }
85
86  void pop(){
87      //check if the stack is empty
88          if(isEmpty()){
89          std::cout<<"Stack underflow."<<std::endl;
90      }
91
92      //display the top value
93      std::cout<<"Popping: "<<stack[top]<<std::endl;
94      //decrement the top value from the stack
95      top--;
96  }
97
98  void printall(){
99      //check if the stack is empty
100     if(isEmpty()){
```

```
100     if(isEmpty()){
101         std::cout<<"Stack underflow."<<std::endl;
102     }
103     std::cout<<"Display Stacks: ";
104     for(int i = top; i >= 0; i--){
105         std::cout<<stack[i]<<" ";
106     }
107
108     std::cout<<std::endl;
109 }
```

OUTPUT CODE

```
Enter the number of max elements for the new stack: 2
====================
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
5.PRINT ALL
6.EXIT
====================
1
New Value to add into the stack
1
====================
```

```
====================
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
5.PRINT ALL
6.EXIT
====================
1
New Value to add into the stack
2
====================
```

```
====================
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
5.PRINT ALL
6.EXIT
====================
1
New Value to add into the stack
3
====================
```

```
====================
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
5.PRINT ALL
6.EXIT
====================
5
Display Stacks: 3 2 1
====================
```

```
====================
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
5.PRINT ALL
6.EXIT
====================
4
0
====================
```

```
====================
STACK OPERATIONS
1.PUSH
2.POP
3.TOP
4.isEmpty
5.PRINT ALL
6.EXIT
====================
6
====================
PS C:\Users\TIPQC\Documents\BALTAZAR\HOA4.2\STACKS>
```

Observation and Analysis:

- First of all, isEmpty function is to check if there are no elements within the stack, we do this by making an if statement and equalizing top to -1, we do not use 0 because that is the first element in our stack.
- Then we have our isFull function where it is used to check if we reach the max capacity of our stack, as usual, we make an if statement having a condition that **top > i-1**, we use i because it is the variable we store in the user input for our max cap, and if top is greater than our max cap, it returns true because the stack is full.
- Next is our push function, where we need to first check if the stack is full as we do not want to overflow the stack and make an error. We then ask the user for a value to add into the stack and store it into the **newData** variable, then from our next line (**stack[++top] = newData**), this adds our value to the top of the stack.
- The top function is for checking what element is in the top of our stack. But before coding the checking part, we always need to check if our stack is empty or not.
- This is the same for pop function as well, this function is to remove the top value in our stack.
- Last function that was used was the printall function which is for displaying all elements within our stack by first checking if our stack is empty or not. Then we display the stacks by using a for loop.
- I also modified the main function to be more ideal to the eyes and added minimal designs as well.

B2. CODE *(main3.cpp)*

```cpp
#include <iostream>
#include "stackList.h"

int main(){
    std::cout<<"===PUSHIN 'c' 'p' 'e' ONTO THE STACK ===\n";
    //push
    push('c');
    push('p');
    push('e');

    //display the current top
    Top<char>(); //Top should be 'e'

    std::cout<<"\n===PRINTING ALL===\n";
    printall<char>();
    std::cout<<"\n===POPPING ALL THE ELEMENTS ===\n";

    while(!isEmpty<char>()){
        std::cout<<"Popped: "<< pop<char>() <<std::endl;
    }

    std::cout<<"\n===TESTING THE UNDERFLOW===\n";
    pop<char>();

    return 0;
}
```

B.2 CODE *(stackList.h)*

```
#ifndef STACK_LIST_H
#define STACK_LIST_H
#include <iostream>

//class node that contains data and the next
template <typename T>
class Node{
    public:
        T data; // Value that we need to store
        Node* next;
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

//push
template <typename T>
```

```
21  void push(T newData){
22      //create a new node and assign the newData
23      Node<T>* newNode = new Node<T>;
24      newNode->data=newData;
25
26      //new node points to current top, then becomes the new top
27      newNode->next = head<T>;
28      head<T> = newNode;
29  }
30
31  //pop
32  template <typename T>
33  T pop(){
34      //check if the stack is empty
35      if(head<T> == nullptr){
36          std::cout<<"stack overflow. "<<std::endl;
37      }
38
39      //save the current top node
40      Node<T>* temp = head<T>;
```

```
41
42      //store its data
43      T tempVal = temp->data;
44
45      //move the head to the next node
46      head<T> = head<T>->next;
47
48      //free the removed node
49      delete temp;
50
51      //return the removed value
52      return tempVal;
53  }
54  //top
55  template <typename T>
56  void Top(){
57      //error catching
58      if(head<T> == nullptr){
59          std::cout<<"stack underflow. "<<std::endl;
60          return;
```

```
61          }
62          //display the top node
63          std::cout<<"Top of Stack: "<< head<T>->data <<std::endl;
64      }
65
66      //isEmpty
67      template <typename T>
68      bool isEmpty(){
69          return head<T> == nullptr;
70      }
71
72      template <typename T>
73      void printall(){
74          if(head<T> == nullptr){
75              std::cout<<"Cannot print if empty "<<std::endl;
76              return;
77          }
78
79          Node<T>* temp = head<T>;
80
81          while(temp != nullptr){
82              std::cout<<temp->data<<" ";
83              temp = temp->next;
84          }
85          std::cout << "\n";
86      }
87
88
89      #endif
```

B2. OUTPUT

```
===PUSHIN 'c' 'p' 'e' ONTO THE STACK ===
Top of Stack: e

===PRINTING ALL===
e p c

===POPPING ALL THE ELEMENTS ===
Popped: e
Popped: p
Popped: c

===TESTING THE UNDERFLOW===
stack overflow.
```

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

CODE *(main3.cpp)*

```cpp
#include <iostream>
#include "stackList.h"

bool isBalanced(char exp[]);

int main(){
    char* exprsn[] = {
        "(A+B)+(C-D)",
        "((A+B)+(C-D)",
        "((A+B)+[C-D])",
        "((A+B]+[C-D]}"};

    for (int i = 0; i < 4; i++){
        char*exp= exprsn[i];
        std::cout<< "\nExpression:"<<exp<<std::endl;
        if (isBalanced(exp) == true){
            std::cout<< "Valid?: YES\n";
        }
        else{
            std::cout <<"Valid?: NO\n";
        }
    }
    return 0;
}

```

```cpp
int length(char exp[]){
    int count = 0;
    for(int i = 0; exp[i] != 0; i++){
        count++;
    }
    return count;
}
// function to check if balanced or nah
bool isBalanced(char exp[]){
    while (!isEmpty<char>())
        pop<char>(); // we need to clear stack to not reuse data from prev expression

    int len = length(exp);
    for(int i = 0; i < len; i++){
        if(exp[i] == '(' || exp[i] == '{' || exp[i] == '[')
            push(exp[i]);

        if(exp[i] == ')' || exp[i] == '}' || exp[i] == ']'){
            if(isEmpty<char>()){
                std::cout<<"Error! no opening symbol for " <<exp[i]<<std::endl;
                return false;
            }
            char top = pop<char>();

            if(exp[i] == ')' && top != '('){
                std::cout <<"Error! "<<top<< " does not match " <<exp[i]<<std::endl;
                return false;
            }
            if(exp[i] == '}' && top != '{'){
                std::cout <<"Error! "<<top<< "does not match " <<exp[i]<<std::endl;
                return false;
            }
            if(exp[i] == ']'&& top != '['){
                std::cout <<"Error! "<<top<< " does not match " << exp[i]<<std::endl;
                return false;
            }
        }
    }
    if (!isEmpty<char>()){
        std::cout << "Error! No closing symbol!\n";
        return false;
    }
    return true;
}
```

CODE *(stackList.h)*

```
#ifndef STACK_LIST_H
#define STACK_LIST_H
#include <iostream>

//class node that contains data and the next
template <typename T>
class Node{
    public:
        T data; // Value that we need to store
        Node* next;
};

//global pointer for the top of the stack
template <typename T>
Node<T> *head = nullptr;

//operations for the stack

//push
template <typename T>
void push(T newData){
    //create a new node and assign the newData
    Node<T>* newNode = new Node<T>;
    newNode->data=newData;

    //new node points to current top, then becomes the new top
    newNode->next = head<T>;
    head<T> = newNode;
}

```

```
31  //pop
32  template <typename T>
33  T pop(){
34      //check if the stack is empty
35      if(head<T> == nullptr){
36          std::cout<<"stack underflow. "<<std::endl;
37          return T();
38      }
39
40      //save the current top node
41      Node<T>* temp = head<T>;
42
43      //store its data
44      T tempVal = temp->data;
45
46      //move the head to the next node
47      head<T> = head<T>->next;
48
49      //free the removed node
50      delete temp;
51
52      //return the removed value
53      return tempVal;
54  }
55  //top
56  template <typename T>
57  void Top(){
58      //error catching
59      if(head<T> == nullptr){
60          std::cout<<"stack underflow. "<<std::endl;
```

```
61          return;
62      }
63      //display the top node
64      std::cout<<"Top of Stack: "<< head<T>->data <<std::endl;
65  }
66
67  //isEmpty
68  template <typename T>
69  bool isEmpty(){
70      return head<T> == nullptr;
71  }
72
73  template <typename T>
74  void printall(){
75      if(head<T> == nullptr){
76          std::cout<<"Cannot print if empty "<<std::endl;
77          return;
78      }
79
80      Node<T>* temp = head<T>;
81
82      while(temp != nullptr){
83          std::cout<<temp->data<<" ";
84          temp = temp->next;
85      }
86      std::cout << "\n";
87  }
88
89
90  #endif
```

OUTPUT

```
Expression:(A+B)+(C-D)
Valid?: YES

Expression:((A+B)+(C-D)
Error! No closing symbol!
Valid?: NO

Expression:((A+B)+[C-D])
Valid?: YES

Expression:((A+B]+[C-D]}
Error! ( does not match ]
Valid?: NO
```

Observation: So my code uses an array that has 4 elements within it as it is the given problem, though initially I wanted to use a cin or a user input but I scrapped that idea as coding each would be much more easier and time efficient as well. So then my int main is mainly used to loop through all four of those elements and prints out if it is balance or not by calling out my function for it. Then under my int main is where another function called length is in, where its general purpose is just to count how many chars are in my exp variable. Then next is my biggest function called isBalanced where its main purpose is to check if all those 4 expressions are balanced or not. So I first added a while loop to clear the whole stack as previously, it kept on doing weird outputs after doing the first or second expression, so I added that stack to remove leftover characters from previous expressions. Then next comes my main loop, which is just going to my length function and going through every character per index. Then next is the if statements where if the start is the given (, [, or {, it will push those onto the stack while others will be just ignored. Next if statement is the checking if we added a closing symbol without adding an opening symbol for it, so it shows an error. Then next is we store our pop in the top variable so that it can still be checked without popping again. Then those 3 if statements are just for checking if each opening or closing entry is a match or not then it outputs an error if it is not and printing as well what kind of it is not a match or not like parenthesis does not match bracket or something. Then after that if statement is another if statement so that if we have an opening symbol and there is no closing symbol, we print out an error message, then if nothing was wrong and no error in the expression it returns true. Nothing much was added on the header file tho, just the pop was added a return T(), so that it could return something on the main cpp file.

SELF CHECKING:

Expression	Valid (Y/N)	Output (Console SS)	Analysis

(A+B)+(C-D)	YES	Expression:(A+B)+(C-D) Valid?: YES	As seen on the output, it matches what we see on the both sides. The parenthesis are equal on A+B and equal on C-D
((A+B)+(C-D)	NO	Expression:((A+B)+(C-D) Error! No closing symbol! Valid?: NO	As seen on the output, we can see that it is not valid because although the inside was alright, we added an opening symbol at the very start without even closing up the parenthesis so it returns with an error.
((A+B)+[C-D])	YES	Expression:((A+B)+[C-D]) Valid?: YES	As seen on the output, we can see that the outer parenthesis is properly balanced and though C-D uses braces, it is well properly balanced.
((A+B]+[C-D]}	NO	Expression:((A+B]+[C-D]} Error! (does not match] Valid?: NO	As seen on the output, the inner part is that a parenthesis and bracket is used to close up A+B which returns an error that says it does not match with each other. So this is why this is not balanced and valid.

CONCLUSION

In this lab activity, I was able to learn about stacks to a certain extent where I was first faced with coding STL which is pretty much easy. Then coding the array stack is my favorite part as it was the main thing I am used to. I was also able to extend my creativity to a certain extent on that array stack code as I added my own touch to the given code. The next thing I faced was making another stack but in linked list, I was able to learn to make a header file for it and although it was pretty difficult at the start, I was able to follow the procedure as it goes on as I was able to familiarize myself with the certain patterns and methods of making a header file from previous lab activities. Then lastly, when my skills were tested in the supplementary activity, during the lab time, I was able to create a concept for it and an initial idea to make my code, although I was faced with multiple errors I needed to debug. But overtime, I was able to fix the errors and also shape my code to what it is now. Overall, I enjoyed this lab especially doing the array stack, it feels like reliving the old school days(last last sem) where I was enjoying myself building only simple code and not that complicated. But, I was able to learn a tons of information and familiarizing with different methods as well.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Claude**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance and debugging on supplementary**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **I used claude to help me out on debugging my first code last week on the main file in my supplementary as it returned an error everytime I run it. AI was not used to replace my original thinking as I already have my own concept to follow as well as it was not used in the analysis for both procedure and supplementary.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.